

ROS2 day1 hw3 레포트

로봇 예비단원 주호진

1. 과제

- turtlesim 패키지를 활용해 그림 그리기

Turtlesim에서 제공하는 토픽 메시지를 활용해 그림 그리는 패키지 제작

ros2 run을 통해 제작한 패키지 실행 시 turtlesim 2d simulator에 그림이 그려지도록 작성

조건

1.WASD 키보드 입력받아 도형 그리기

2.삼각형, 사각형, 원 등 그리도록 작성

3.도형마다 굵기, 색상 다르게 설정

2. 코드

코드는 패키지 기본 세팅을 제외한 부분을 기재했다.

```
import sys
import select
import tty
import termios
import threading

from geometry_msgs.msg import Twist
from turtlesim.srv import SetPen
import sys
```

터틀봇을 제어하기 위한 geometry_msgs와 펜 색상과 굵기를 제어하기 위해 turtlesim.srv를 가져오고, 키입력을 받기 위한 라이브러리를 import해준다.

```

def __init__(self):
    super().__init__('turtlesim_control')
    self.publisher_ = self.create_publisher(Twist, 'turtle1/cmd_vel', 10)

    self.control_timer = self.create_timer(1.0, self.control_loop_callback)

    self.i = 0
    self.current_mode = None
    self.is_running = True

    self.keyboard_thread = threading.Thread(target=self.keyboard_listener_loop)
    self.keyboard_thread.daemon = True
    self.keyboard_thread.start()

    self.pen_cli = self.create_client(SetPen, '/turtle1/set_pen')

```

명령어를 전달해주기 위해 Node 라이브러리에 있는 create_pub과 create_cli를 사용해서 설정하여 준다.

추가적으로 키보드 입력을 받아 그림을 수행하기 위한 변수도 선언해준다.

```

def circle(self):
    if self.i == 0:
        self.set_pen(0, 255, 0, 4)

    msg = Twist()
    msg.linear.x = 2.0 # 전진 속도
    msg.angular.z = 2.0 # 회전 속도
    self.publisher_.publish(msg)
    self.get_logger().info(f'Publishing: "linear.x={msg.linear.x}, angular.z={msg.angular.z}"')

    if self.i > 4:
        stop_msg = Twist()
        self.publisher_.publish(stop_msg)
        self.current_mode = None
        self.i = 0
    self.i += 1

```

```

def rectangle(self): def triangle(self): def pentagon(self):

```

위 4개의 함수는 각각 원, 삼각형, 사각형, 오각형을 그리는 함수이다.

과제 1번을 수행할 때 사용했던 topic의 내용을 code상으로 구현하였다.

Code 구조는 4개가 거의 동일하다.

```

def set_pen(self, r, g, b, width, off=0):
    if not self.pen_cli.wait_for_service(timeout_sec=0.1):
        return
    req = SetPen.Request()
    req.r = r
    req.g = g
    req.b = b
    req.width = width
    req.off = off

    self.pen_cli.call_async(req)

```

펜의 r, g, b와 굵기를 설정할 수 있는 함수이다.

```

def keyboard_listener_loop(self):
    # 스레드 시작 시 터미널 설정을 한 번만 저장하고 Raw 모드로 변경
    settings = termios.tcgetattr(sys.stdin.fileno())
    try:
        tty.setraw(sys.stdin.fileno())
        while self.is_running and rclpy.ok():
            # 0.1초 단위로 키보드 입력이 있는지 확인
            rlist, _, _ = select.select([sys.stdin], [], [], 0.1)
            if rlist:
                key = sys.stdin.read(1)

                # 현재 도형을 그리고 있지 않을 때만 새 명령 받기
                if self.current_mode is None and key in ['w', 'a', 's', 'd']:
                    self.current_mode = key
                    self.i = 0
            finally:
                # 스레드가 끝날 때 터미널 설정을 원래대로 복구
                termios.tcsetattr(sys.stdin.fileno(), termios.TCSADRAIN, settings)

def control_loop_callback(self):
    if self.current_mode is None:
        return

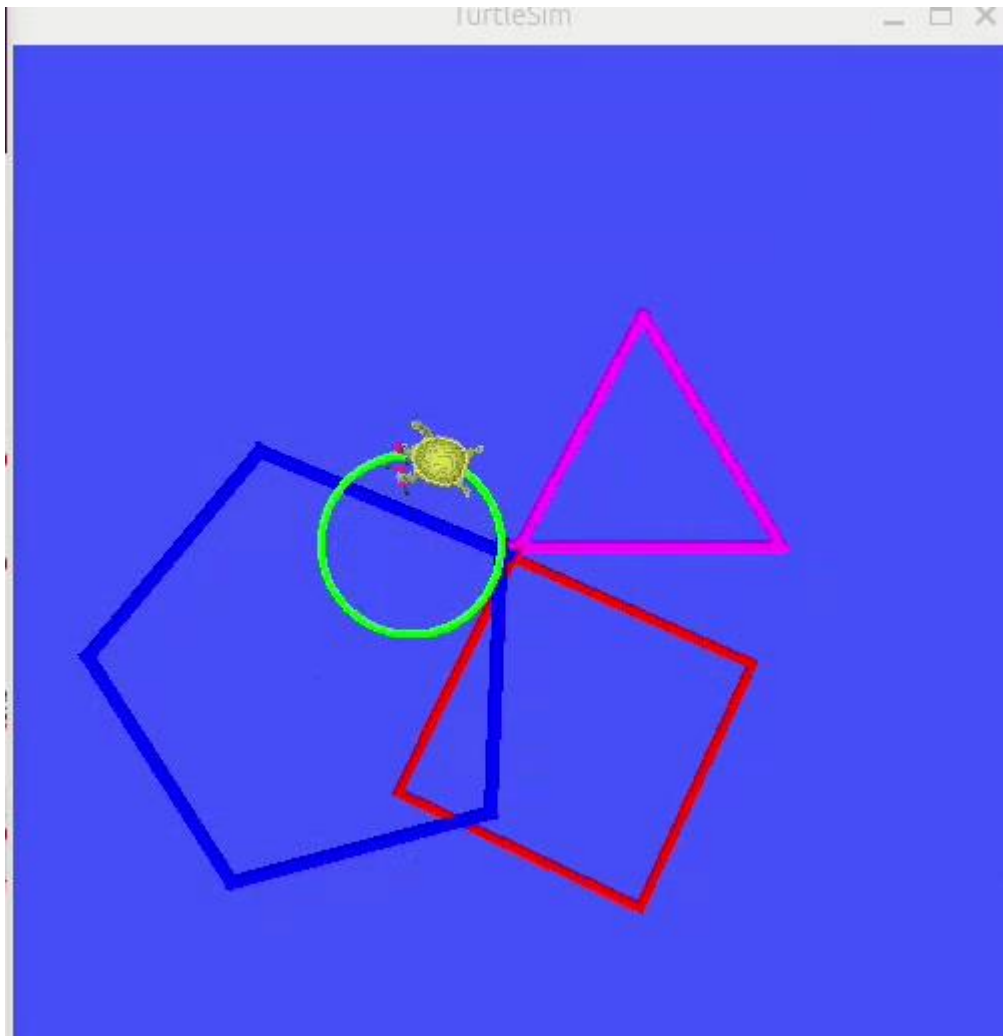
    if self.current_mode == 'w':
        self.circle()
    elif self.current_mode == 'a':
        self.triangle()
    elif self.current_mode == 's':
        self.rectangle()
    elif self.current_mode == 'd':
        self.pentagon()

```

Listener에서 키 입력을 받아 위에서 설정했던 도형을 그리는 함수를 호출해주는 역할을 한다.

3. 작동 영상

작동 영상



4. AI 보고서

<https://share.gemini.google/lt4K0tEGc40O>

키보드 입력을 받아오는 데에 사용했다.

지금 타이머로 받아오고 있는데 이걸 키입력으로 바꿀건데 init에 while로 씌워도 되나

텍스트 보기

아니오, `__init__` 내부에 `while` 루프를 넣는 것은 **노드의 초기화와 실행을 멈추게 하므로(Blocking) 절대 사용하면 안 됩니다.**

처음에 init에 while문으로 키입력을 받아도 되는가 싶어서 질문을 했다가, 키입력 라이브러리를 찾아서 적용해보았다.

`twist`
`기 "keyl`

d | 령게 뜨는데 이거 뭐야 import가 안되는건가

keyboard 말고 다른걸로 키입력 감지 구현 가능한가

네, `keyboard` 라이브러리 대신 **관리자(`sudo`) 권한이 필요 없는** 다른 방법들로 키 입력 감지를 구현할 수 있습니다.

python 키입력 구현을 keyboard라는 라이브러리를 사용한 정보가 많아서 이를 사용했는데, 권한 때문에 실행이 자꾸 안되어서 keyboard를 사용하지 않는 것으로 바꾸었다.

키를 아예 인지를 못하는데 이유가 뭐야

사진을 보니 터미널 창에 `asdwawdasd...` 처럼 입력한 키보드 글자가 그대로 화면에 찍히고 있네요!
이 현상이 바로 **키 인지가 안 되는 핵심 이유**입니다.

🔥 원인 분석

작성하신 코드의 `get_key()` 함수는 터미널을 입력 즉시 반응하는 'Raw 모드'로 바꿨다가, 키가 없으면 곧바로 엔터를 쳐야만 입력되는 '기본(Cooked) 모드'로 되돌리는 작업을 0.01초마다 아주 빠르게 반복하고 있습니다.

결과적으로 터미널이 99%의 시간 동안 '기본 모드'에 머물러 있게 됩니다. 이때 키를 누르면 파이썬이 키를 낚아채지 못하고, 터미널 버퍼에 글자가 쌓이면서 화면에 출력만 된 채 엔터 키를 기다리게 되는 것입니다.

바꿨을 때 키 감지가 안되어서 질문을 했었는데 key를 받을 때 raw와 cooked이 있는데 cooked로 설정 시 enter를 쳐야만 적용된다는 것을 알았다.

turtlesim 움직이는 궤적 색상 변경

turtlesim에서 거북이가 움직일 때 남기는 궤적(선)의 색상은 `/turtle1/set_pen` 서비스를 호출하여 변경할 수 있습니다.  GitHub +1

set pen은 퍼블리셔 안써?

결론부터 말씀드리면, `set_pen` 은 퍼블리셔를 아예 쓰지 않습니다. 오직 서비스 클라이언트 (Service Client)로만 작동하도록 만들어져 있습니다.

Set pen을 사용하기 위해 앞서 구현했던 pub, sub이 아닌 서비스 클라이언트로 작동한다는 것을 알았다.